

BibleApp Backend

API contract design, testing approach, and feature roadmap for the Ktor + Exposed + PostgreSQL + JWT backend (branch: KtorFeature)

This backend exists to support two roadmap items from the app: **user accounts with cross-device sync** (bookmarks, reading progress, notes) and, eventually, the Bible chatbot. The Gradle scaffold for the server is already in place under the new `:server` module. This document covers the two things worth doing *before* writing real endpoints, and lays out the order to build the rest in.

1. Design the API contract before implementation

You've already done this once, informally, for the Bible content sources — `HelloAoDtos.kt` and `OurMannaDailyVerseResponseDto.kt` are DTOs that mirror someone else's API shape. This time you're the API designer, not just the consumer, so the shape has to be decided deliberately instead of copied from a response you received. Writing it down first (even a short markdown file) prevents the shape from drifting as you build each endpoint separately.

Endpoints

Group endpoints by resource, not by screen. The two roadmap features map to two resource groups:

- **Auth** — `/api/v1/auth/register`, `/api/v1/auth/login`, `/api/v1/auth/refresh`
- **Sync** — `/api/v1/bookmarks`, `/api/v1/reading-progress`, `/api/v1/notes` (standard REST verbs: GET list, POST create, PUT/PATCH update, DELETE remove)

This is exactly why `AuthRoutes.kt` and `SyncRoutes.kt` already exist as separate empty files in the scaffold — the grouping is decided, the contents are not.

Request / response DTOs

Same pattern as your existing remote DTOs: one Kotlin `@Serializable` data class per request body and per response body, kept separate from your internal Exposed table/entity classes. Never return a database row type directly from a route — it couples your wire format to your schema, and you lose the ability to change one without breaking the other.

```
@Serializable
data class RegisterRequest(val email: String, val password: String)

@Serializable
data class AuthResponse(val userId: String, val accessToken: String, val expiresInSeconds: Long)
```

Decide field names, nullability, and which fields are optional up front. Changing a field from required to optional later is easy; the reverse breaks the app.

Versioning

Put the version in the URL path from day one: `/api/v1/ . . .` It's the simplest scheme, it's visible in every log line and every `curl` command, and it means a future breaking change (v2) can run side-by-side with v1 while old app installs finish updating — important for a mobile client you don't fully control the update cadence of.

Error-response shape

Right now `plugins/Router.kt` installs `StatusPages` with a placeholder `ErrorResponse(val error: String)` (in `models/ErrorResponse.kt`). Before writing real endpoints, decide the actual shape every error will use — the same envelope for validation errors, auth failures, and server errors, so the Android client only ever needs one error-parsing code path. A shape that scales past a single message field:

```
@Serializable
data class ErrorResponse(
    val code: String,           // e.g. "INVALID_CREDENTIALS", "VALIDATION_ERROR"
    val message: String,       // human-readable, safe to show or log
    val fieldErrors: Map<String, String?> = null // for validation failures
)
```

Map exceptions to specific HTTP status codes in `StatusPages` (400 for validation, 401 for bad/missing auth, 404 for not found, 409 for conflicts, 500 for anything unexpected) rather than returning 500 for everything — the current scaffold's `exception<Throwable>` catch-all is a starting point, not the final handler.

Before writing the first real route

- List every endpoint (method + path) for the feature you're about to build.
- Write the request and response DTO for each one, including error cases.
- Decide status codes for the happy path and each failure mode.
- Only then start implementing — the route bodies become straightforward once the shapes are fixed.

2. Test with Ktor's testApplication engine

Ktor ships an in-process test engine that spins up your real Application module — routing, serialization, auth, everything except an actual DB and actual open socket — and gives you an HTTP client to call it. It's faster than manual curl/Postman checks, it's repeatable, and it survives refactors because it exercises the same route/plugin pipeline the real server uses.

This is already wired up: `server/src/test/kotlin/.../ApplicationTest.kt` boots the whole module and asserts `GET /health` returns 200. Use it as the template for every new route.

```
class ApplicationTest {
    @Test
    fun healthCheckReturnsOk() = testApplication {
        environment { config = ApplicationConfig("application.conf") }
        val response = client.get("/health")
        assertEquals(HttpStatusCode.OK, response.status)
    }
}
```

One catch already resolved in the scaffold: `testApplication` does not read `application.conf` automatically — you must point `environment.config` at it explicitly, as shown above. Skip that line and every call to `environment.config.property(...)` inside your plugins (JWT secret, DB URL) throws instead of finding a value.

What to test at each layer

- **Routes** — status code + response body shape for the happy path and each documented error case (missing field, wrong password, not found).
- **Auth** — a route behind `authenticate("auth-jwt")` rejects requests with no token or an expired/invalid token (401), and accepts a validly-signed one.
- **Serialization** — a request body missing an optional field still parses; an unknown extra field doesn't break parsing (covered by `ignoreUnknownKeys = true` already set in `Serialization.kt`).

Once real DB-backed routes exist, point the test's storage config at a throwaway database — an in-memory H2 instance for pure-unit-style tests, or (closer to production behavior) a real Postgres via a Testcontainers container — rather than the dev database in `application.conf`. That's a decision to make once persistence lands (Phase 3 below), not before.

3. Feature roadmap

Suggested build order. Each phase produces something runnable and testable before the next one starts — don't jump ahead to sync endpoints before auth actually issues a working token, and don't wire real Postgres before the schema is decided.

Phase	Goal	Key files
0 — Scaffold (done)	Gradle module, plugin wiring (serialization, JWT skeleton, DB connection skeleton, routing + StatusPages), health check, first passing test.	:server module; plugins/*.kt; ApplicationTest.kt
1 — Contract design	Write down every auth + sync endpoint, its request/response DTOs, and the error shape (Section 1 of this doc). No server code yet.	A short design note (markdown, or in the PDF's ideas)
2 — Auth (JWT)	Implement register/login/refresh against a real password hash (e.g. bcrypt/argon2), issue signed JWTs, fill in the real claim validation in Security.kt.	routes/AuthRoutes.kt; plugins/Security.kt
3 — Persistence	Define Exposed tables (Users, plus one per synced resource), connect to a real Postgres (local Docker container first), replace SchemaUtils.create with a real migration tool once the schema stabilizes.	db/tables/*.kt; plugins/Databases.kt
4 — Sync endpoints	Bookmarks, reading progress, notes: CRUD behind authenticate("auth-jwt"), scoped to the authenticated user's own data.	routes/SyncRoutes.kt
5 — Hardening & deploy	Real error-code mapping in StatusPages, request validation, rate limiting on auth routes, structured logging, environment-based secrets, pick a host (Railway/Fly.io/VPS), CI for the :server module.	plugins/Router.kt; application.conf; CI config
6 — Android integration	Point HttpClientProvider at the deployed backend, add token storage (EncryptedSharedPreferences/DataStore), wire login/register UI, replace local-only bookmarks/progress/notes with synced versions.	data/remote/ HttpClientProvider.kt

Each phase after 0 should start by extending the tests from Section 2 before writing the feature — write the testApplication case for the new route (or the new authenticate(...) check) first, watch it fail, then implement.

What's already in the repository

- settings.gradle.kts — :server module registered alongside :app.
- gradle/libs.versions.toml — Ktor 3.0.3, Exposed 0.61.0, PostgreSQL driver 42.7.7, Logback 1.6.3 (versions chosen to match the project's existing Kotlin 2.0.21 — newer Ktor/Exposed releases require a newer Kotlin compiler than the app module currently uses).
- server/build.gradle.kts — plain Kotlin/JVM module, independent of the Android Gradle Plugin.
- server/src/main/kotlin/.../Application.kt — entry point (EngineMain) wiring all plugins together.

- `plugins/Serialization.kt`, `Security.kt`, `Databases.kt`, `Routing.kt`, `Monitoring.kt` — one file per concern, matching the structure the official Ktor project generator produces.
- `routes/AuthRoutes.kt`, `routes/SyncRoutes.kt` — empty extension functions, already wired into routing, ready for Phase 2 and Phase 4.
- `models/ErrorResponse.kt` — placeholder error shape, to be replaced per Section 1.
- `src/main/resources/application.conf` — port, JWT, and storage settings, each overridable by an environment variable (`PORT`, `JWT_SECRET`, `DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`) so the dev secret never has to be the production secret.
- `src/test/kotlin/.../ApplicationTest.kt` — passing `testApplication` smoke test.

Verified: `./gradlew :server:test` and `./gradlew :app:compileDebugKotlin :server:build` both pass on this scaffold.